

Parallel and Distributed Databases

Syllabus Content

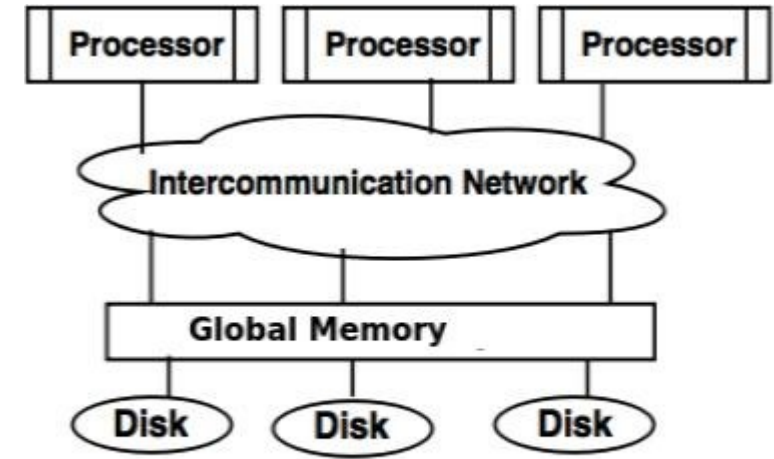
- **Parallel Database:**
 - Architecture, I/O Parallelism, Interquery, Intraquery
 - Intraoperation and Interoperation Parallelism
- **Distributed Databases**
 - Types of Distributed Database Systems,
 - Distributed Data Storage, Distributed Query Processing

Parallel Database

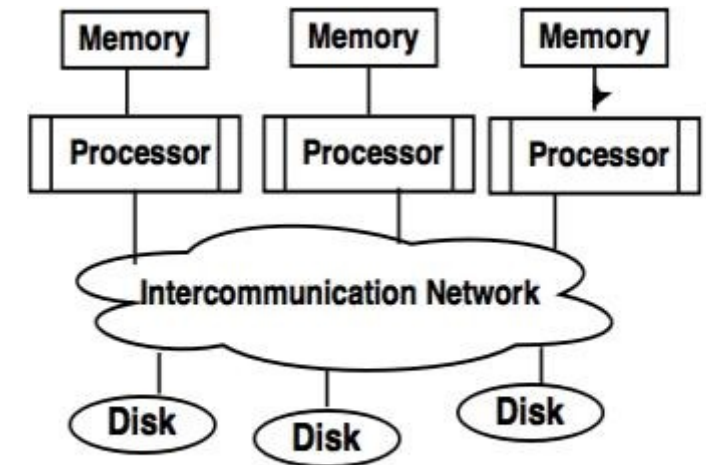
- Parallel DBMS is a Database Management System **that runs through multiple processors and disks.**
- **They combine two or more processors also disk storage** that helps make operations and executions easier and faster.
- They are **designed to execute concurrent operations.**
- **Architectural Models**
- There are several architectural models for parallel Database, which are given below –
- **Shared memory architecture.**
- **Shared disk architecture.**
- **Shared nothing architecture.**

Parallel Database

- **Shared Memory System**
- Every **computer processor is able to access and process data from multiple memory modules** or unit through intercommunication channel.
- This architecture is also commonly known as SMP or Symmetric Multi-processing
- **Shared Disk System**
- A Shared Disk System is an architecture of Database Management System where **every computer processors can access multiple disk through intercommunication network**.
- It can also access and utilize every local memory.



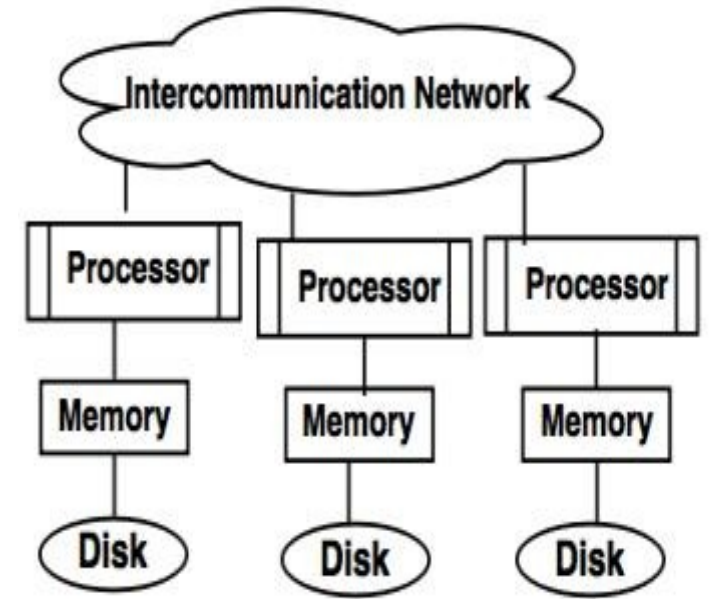
Shared Memory System in Parallel Databases



Shared disk system in Parallel Databases

Parallel Database

- **Shared Nothing System**
- A Shared Nothing System is an architecture of Database Management System where **every processor has their own disk and memory for the objective of efficient workflows.**
- The processors can communicate with other processors using intercommunication network.
- Each of the processors act like servers to store data on the disk.



Shared nothing disk system in Parallel Databases

I/O parallelism in parallel database

- I/O parallelism refers to **reducing the time required to retrieve relations from disk by partitioning the relations on multiple disks.**
- The most common form of data partitioning in a parallel database environment is horizontal partitioning.
- In horizontal partitioning, the tuples of a relation are divided (or declustered) among many disks, so that each tuple resides on one disk.

•Partitioning Techniques

- Three basic data-partitioning strategies. Assume that there are n disks,
- $D_0, D_1, \dots, D_{n-1}$, across which the data are to be partitioned.

•Round-robin.

- This strategy scans the relation in any order and sends the i th tuple to disk number $D_{i \bmod n}$.
- The **round-robin scheme ensures an even distribution of tuples across disks;** that is, each disk has approximately the same number of tuples as the others.

I/O parallelism in parallel database

- Hash partitioning.

- This declustering strategy designates one or more attributes from the given relation's schema as the partitioning attributes.

- A hash function is chosen whose range is $\{0, 1, \dots, n - 1\}$. Each tuple of the original relation is hashed on the partitioning attributes. If the hash function returns i , then the tuple is placed on disk D_i .

- Range partitioning.

- This strategy distributes contiguous attribute-value ranges to each disk. It chooses a partitioning attribute, A , as a **partitioning vector**.

- The relation is partitioned as follows. Let $[v_0, v_1, \dots, v_{n-2}]$ denote the partitioning vector, such that, if $i < j$, then $v_i < v_j$. Consider a tuple t such that $t[A] = x$. If $x < v_0$, then t goes on disk D_0 . If $x \geq v_{n-2}$, then t goes on disk D_{n-1} . If $v_i \leq x < v_{i+1}$, then t goes on disk D_{i+1} .

- For example, range partitioning with three disks numbered 0, 1, and 2 may assign tuples with values less than 5 to disk 0, values between 5 and 40 to disk 1, and values greater than 40 to disk 2.

Interquery and Intraquery parallelism

- **Interquery Parallelism**
- In interquery parallelism, **different queries or transaction execute in parallel with one another.**
- This form of **parallelism can increase transactions throughput.** The response times of individual transactions are not faster than they would be if the transactions were run in isolation.
- Thus, the **primary use of interquery parallelism is to scale up a transaction processing system to support a more significant number of transactions per second.**
- Interquery parallelism is the **easiest form of parallelism to support in a database system—particularly in a shared-memory parallel system.**
- Database systems designed for single-processor systems can be used with few or no changes on a shared-memory parallel architecture, since even sequential database systems support concurrent processing.

Interquery and Intraquery parallelism

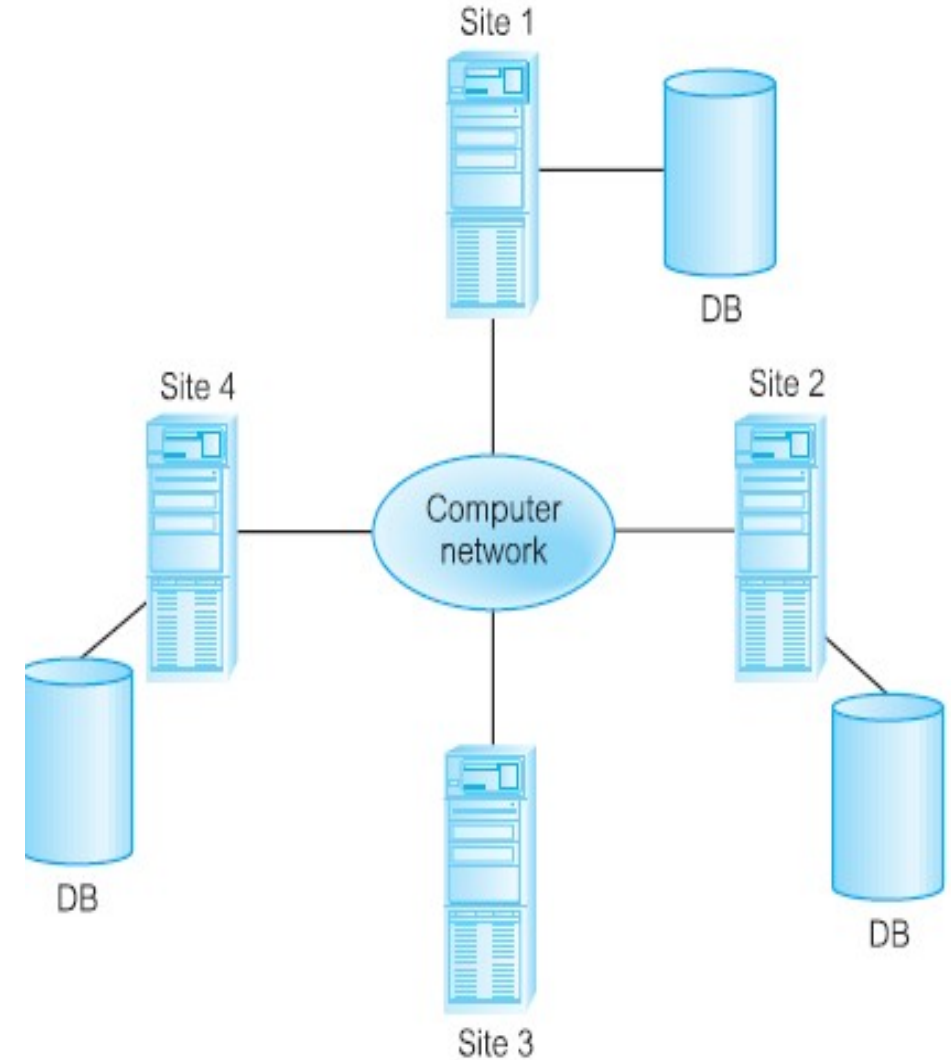
- Intraquery Parallelism
- Intraquery parallelism **defines the execution of a single query in parallel on multiple processors and disks.**
- Using intraquery parallelism **is essential for speeding up long-running queries.**
- This application of parallelism decomposes the serial SQL query into lower-level operations such as scan, join, sort, and aggregation.
- To illustrate the parallel evaluation of a query, **consider a query that requires a relation to be sorted.** Suppose that the relation has been partitioned across multiple disks by range partitioning on some attribute, and the sort is requested on the partitioning attribute. **The sort operation can be implemented by sorting each partition in parallel, then concatenating the sorted partitions to get the final sorted relation.**

Intraoperation and Interoperation Parallelism

- we may be able to pipeline the output of one operation to another operation. The two operations can be executed in parallel on separate processors,
- one generating output that is consumed by the other, even as it is generated.
- In summary, the **execution of a single query can be parallelized in two ways:**
- **Intraoperation parallelism.**
- We can speed up processing of a query by parallelizing the execution of each individual operation, such as sort, select, project, and join.
- **Interoperation parallelism.**
- We can speed up processing of a query by executing in parallel the different operations in a query expression.

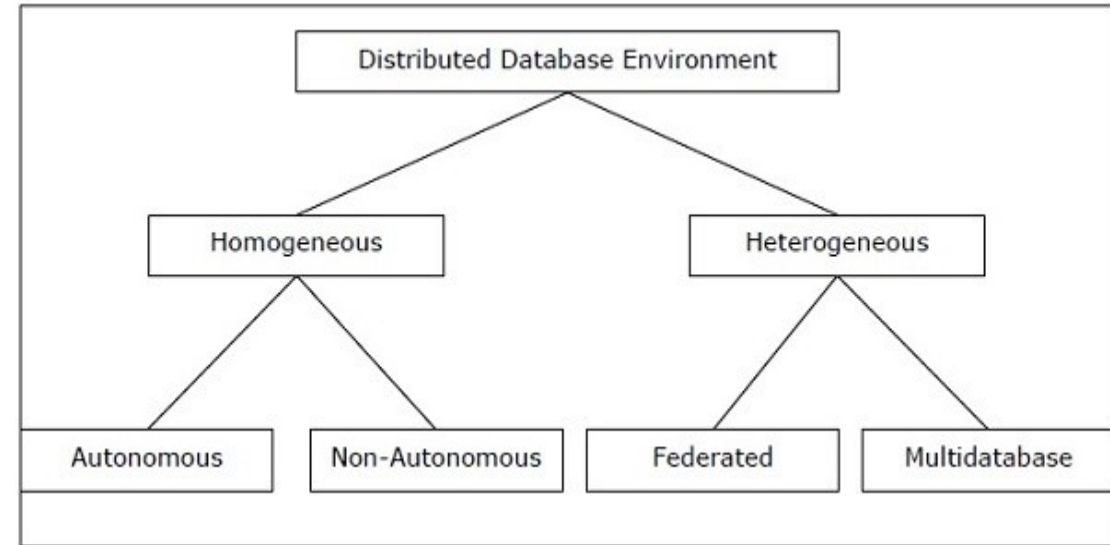
Distributed Databases

- **What is a distributed database?**
- *Distributed database system is one in which the **data belonging to a single logical database is distributed to two or more physical databases to ensure reliability and availability***
- A distributed database is a database in which all storage devices are not attached to a common CPU. **Data may be stored in multiple sites separate from each other.**
- In a distributed database, the data is spread or replicated among several databases which are physically separate from each other. **These databases are connected through a network so that they appear as a single database to the user.**



Types of Distributed Databases

- Distributed databases can be broadly classified into **homogeneous** and **heterogeneous distributed database** environments
- **Homogeneous Distributed Databases**
- In a homogeneous distributed database, **all the sites use identical DBMS and operating systems**. Its properties are
- The **sites use very similar software**.
- The **sites use identical DBMS or DBMS from the same vendor**.
- Each site is **aware of all other sites and cooperates with other sites to process user requests**.
- The database is **accessed through a single interface as if it is a single database**.

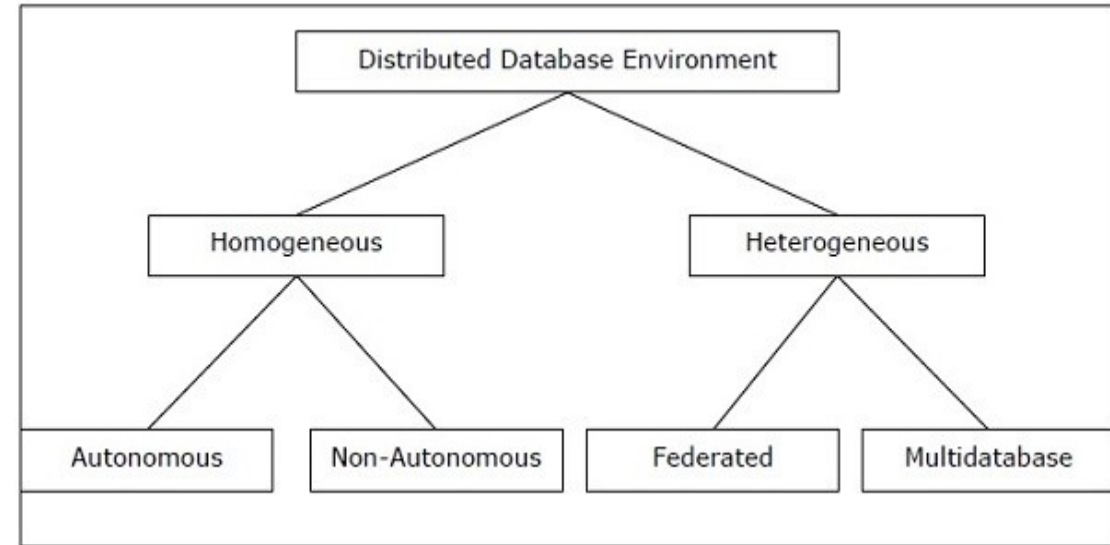


Types of Distributed Databases

- There are two types of homogeneous distributed database are:

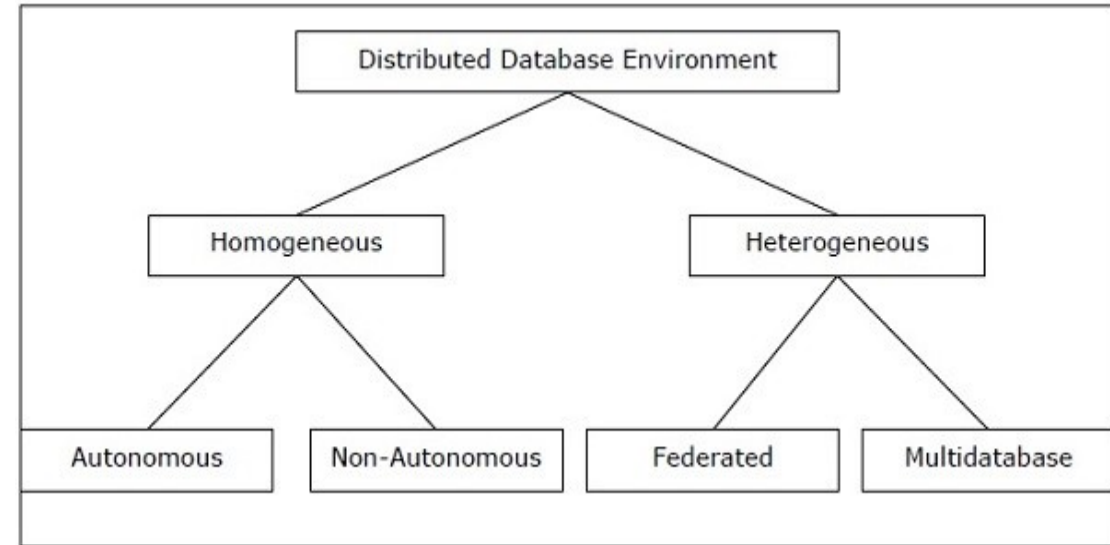
1. Autonomous – Each database is independent that functions on its own. They are integrated by a controlling application and use message passing to share data updates.

2. Non-autonomous – Data is distributed across the homogeneous nodes and a central or master DBMS co-ordinates data updates across the sites.



Types of Distributed Databases

- **Heterogeneous Distributed Databases**
- In a heterogeneous distributed database, **different sites have different operating systems, DBMS products and data models.** Its properties are –
- Different **sites use dissimilar schemas and software.**
- The system may be composed of a **variety of DBMSs like relational, network, hierarchical or object oriented.**
- **Query processing is complex due to dissimilar schemas.**
- **Transaction processing is complex due to dissimilar software.**
- A site **may not be aware of other sites and so there is limited co-operation in processing user requests.**

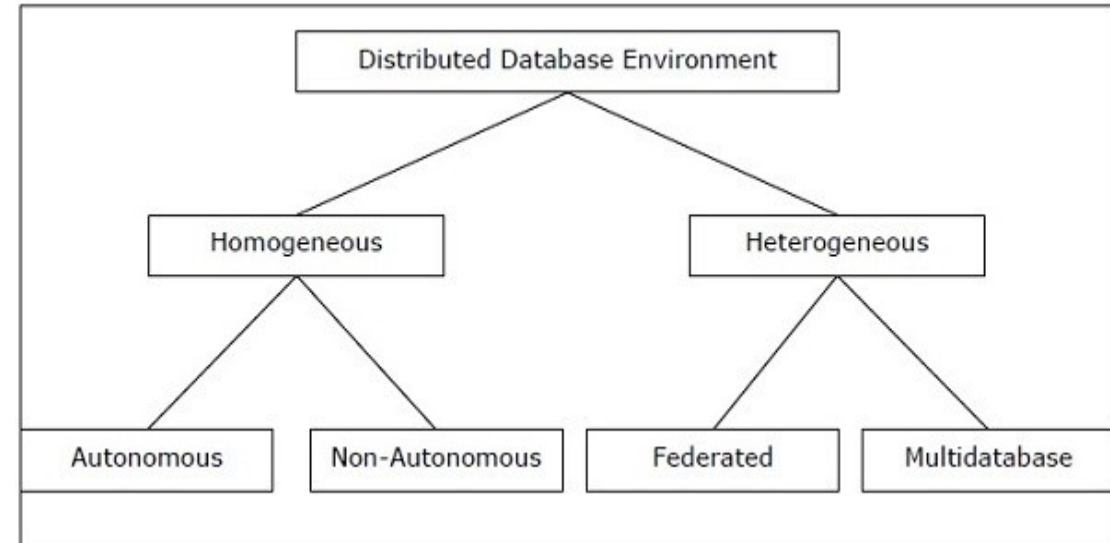


Types of Distributed Databases

- Types of Heterogeneous Distributed Databases

1.Federated – The heterogeneous database **systems are independent in nature and integrated together so that they function as a single database system.**

2.Un-federated – The database systems employ a **central coordinating module through which the databases are accessed.**



Distributed Data Storage

- Distributed Data storage is an intelligent **distribution of your data pieces, (called data fragments)** to improve database performance and Data Availability for end-users.
- It **aims to reduce overall costs of transaction processing** while also providing accurate data rapidly in your DDBMS systems.
- **Distributed Data storage is one of the key steps** in building your Distributed Database Systems.
- There are two common strategies used in optimal Data Allocation: **Data Fragmentation and Data Replication.**

Distributed Data Storage

- **Fragmentation –**

In this approach, the relations are **fragmented (i.e., they're divided into smaller parts)** and each of the fragments is stored in different sites where they're required.

- Fragmentation is a process of **disintegrating relations or tables into several partitions in multiple sites**. It divides a database into various subtables and sub relations so that data can be distributed and stored efficiently. Fragmentation of relations can be done in two ways:

- **Horizontal fragmentation- Splitting by rows** – The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.

- For example, in the student schema, if the details of all students of Computer Science Course needs to be maintained at the School of Computer Science, then the designer will horizontally fragment the database as follows –

- **CREATE COMP_STD AS**

- **SELECT * FROM STUDENT**

- **WHERE COURSE = "Computer Science";**

Distributed Data Storage

- **Vertical fragmentation – Splitting by columns –**
- The schema of the relation is divided into smaller schemas. **Each fragment must contain a common candidate key so as to ensure a lossless join.**
- In certain cases, an approach that is hybrid of fragmentation and replication is used.
- For example, let us consider that a University database keeps records of all registered students in a Student table having the following schema.

- STUDENT

Regd_No	Name	Course	Address	Semester	Fees	Marks
---------	------	--------	---------	----------	------	-------

- Now, the fees details are maintained in the accounts section. In this case, the designer will fragment the database as follows –
- **CREATE TABLE STD_FEES AS**
- **SELECT Regd_No, Fees**
- **FROM STUDENT;**

Distributed Data Storage

- Hybrid Fragmentation**

- In hybrid fragmentation, a combination of horizontal and vertical fragmentation techniques are used.

- Hybrid fragmentation can be done in two alternative ways –**

- At first, generate a set of horizontal fragments; then generate vertical fragments from one or more of the horizontal fragments.

- At first, generate a set of vertical fragments; then generate horizontal fragments from one or more of the vertical fragments.

Distributed Data Storage

•Fragmentation Example

This is global Table

CustomerID	CustomerName	City	Gender
1	Bob	Mumbai	Male
2	Alice	Bangalore	Female
3	Milind	Agra	Male
4	Jaya	Pune	Female

Fragment 1

CustomerID	CustomerName	City	Gender
1	Bob	Mumbai	Male
3	Milind	Agra	Male

Horizontal Fragment

Fragment 2

CustomerID	CustomerName	City	Gender
2	Alice	Bangalore	Female
4	Jaya	Pune	Female

Fragment 1

CustomerID	CustomerName
1	Bob
2	Alice
3	Milind
4	Jaya

Vertical Fragment

Fragment 2

CustomerID	City	Gender
1	Mumbai	Male
2	Bangalore	Female
3	Agra	Male
4	Pune	Female

Distributed Data Storage

- **Replication –**

In this approach, **the entire relationship is stored redundantly at 2 or more sites**. If the entire database is available at all sites, it is a fully redundant database. **Hence, in replication, systems maintain copies of data.**

- This is advantageous as it increases the availability of data at different sites.
- However, it has certain disadvantages as well. **Data needs to be constantly updated.** Any change made at one site needs to be recorded at every site that relation is stored or else it may lead to inconsistency. This is a lot of overhead. Also, concurrency control becomes way more complex as concurrent access now needs to be checked over a number of sites.

- **Advantages of Data Replication**

- **Reliability** – In case of failure of any site, the database system continues to work since a copy is available at another site(s).
- **Reduction in Network Load** – Since local copies of data are available, query processing can be done with reduced network usage, particularly during prime hours. Data updating can be done at non-prime hours.
- **Quicker Response** – Availability of local copies of data ensures quick query processing and consequently quick response time.
- **Simpler Transactions** – Transactions require less number of joins of tables located at different sites and minimal coordination across the network. Thus, they become simpler in nature.

Distributed Data Storage

- Types of Data Replication In DBMS

- Transactional Replication**

- Snapshot Replication**

- Merge Replication**

- Transactional Replication**

•Transactional Replication makes a complete copy of your database, as well as copies of new data changes. In this type of Data Replication, **changes to your database are synced in real-time and in the same order as they occur**. This guarantees transactional consistency.

- Snapshot Replication**

•Snapshot Replication is perhaps the simplest type of Data Replication that copies “snapshots” of your database. **It replicates the current state of your database as is, at a specific point in time, without including any changes/updates to your data**. This kind of replication is helpful when changes made to your databases are infrequent.

- Merge Replication**

•**Merge Replication combines data from several databases into a single database**. This type of Data Replication tracks subsequent data changes and schema modifications made at publishers and subscribers and synchronizes the same to your database using merge agents. A great advantage of using Merge Replication is that it allows publishers and subscribers to independently modify the database.